

How to add the next case study

Written while I still remember how the site works, so the next case is a 15-minute job instead of an archaeology dig.

Where it goes

File: `~/projects/ys-home/index.html` **Section:** `<!-- ===== work ===== -->`, inside `<section class="reveal">`

Cases are plain `<article class="card">` blocks, stacked in order. **Order is the argument:** shipped work with numbers goes first, designed-but-not-built goes last. Insert accordingly — don't just append.

Screenshots go in `~/projects/ys-home/assets/`.

The three beats

Every case follows the same shape, same as Week 2:

1. **Problem** — what was actually broken, in one short paragraph. Concrete, not "businesses need better tooling."
2. **What I did** — the mechanism. Name the real technique. This is where the specificity lives, and it's the part reviewers said made the work credible.
3. **What came of it** — numbers if they exist. If they don't, say so and lead with mechanism instead. Never invent a metric to fill the pattern.

Steps

1. **Take one screenshot** that proves the thing exists and works. Save it to `assets/`. Optimize it before committing:

```
bash cd ~/projects/ys-home/assets python3 -c " from PIL import Image im = Image.open('NEW-SHOT.png').convert('RGB') if im.width > 1400: im = im.resize((1400, round(im.height*1400/im.width))), Image.LANCZOS) im.quantize(colors=256).save('NEW-SHOT.png', optimize=True) "
```

1. **Copy the template below** into the work section, in the right position.
2. **Set the status chip honestly.** Three variants exist:
3. `<span class="status">Live · real users</span>` — green, genuinely shipped
4. `<span class="status wip">Deployed · testnet</span>` — amber, partial
5. `<span class="status internal">Internal tool · one user (me)</span>` — grey

6. **Check it locally** before deploying:

```
bash cd ~/projects/ys-home && python3 -m http.server 8000 # open http://localhost:8000 — check desktop AND a narrow window
```

1. **Ship it:**

```
bash cd ~/projects/ys-home git add -A && git commit -m "Add <name> case study" bash build-deploy.sh # writes ~/projects/ys-home-dist # drag ~/projects/ys-home-dist onto app.netlify.com/drop
```

Always deploy `ys-home-dist`, never `ys-home` — the build script exists so `.git` never gets uploaded.

1. **Optional:** add the longer write-up to the case-study site at `~/projects/ys-dev/` (a page like `fl04.html`, plus a card on its `index.html`), then `git push` — that one deploys itself via GitHub Pages.

Paste-ready template

```
<article class="card">
  <div class="card-head">
    <h3>PROJECT NAME</h3>
    <span class="status">Live · real users</span>
  </div>
  <p>
    PROBLEM — what was broken, concretely.
  </p>
  <p>
    WHAT I DID — the actual mechanism, named specifically.
  </p>
  <div class="metrics">
```

```

    <div class="metric"><strong>NUM</strong><span>what it counts</span></div>
    <div class="metric"><strong>NUM</strong><span>what it counts</span></div>
  </div>
  <a class="shot" href="assets/NEW-SHOT.png" target="_blank" rel="noopener">
    
  </a>
  <p class="caption">One line on what the screenshot shows.</p>
  <ul class="tags">
    <li>Tech</li><li>Tech</li><li>Tech</li>
  </ul>
</article>

```

Delete the `<div class="metrics">` block entirely if there are no real numbers. An empty metrics row looks worse than none.

Two rules I keep breaking

- **Numbers live on the CV before they reach the site.** The reviewer's hardest hit was that CampusPQ's "5,000+ questions, 98%" sat on my CV for months and never made it onto the page. When a project gets a metric, put it here the same day.
- **Say what's shipped vs. designed.** The status chips exist for this. Overclaiming is the fastest way to lose a technical reader.

Next case: DevMemory

Repo: `~/projects/devmemory` — <https://github.com/Yuguda999/devmemory> **Published:** <https://pypi.org/project/devmemory/>

Why this one: it's real shipped software with a public install path anyone can verify, and it's the closest thing I have to pure infrastructure — which is exactly the positioning the site claims.

Three beats, roughly:

- **Problem:** switching AI coding tools (Cursor → Claude → Windsurf) kills your context. You re-explain the goal, re-paste the same snippets, lose momentum every time.
- **What I did:** an MCP server that persists coding context across tools — self-hostable with no external API dependency, with JWT/API-key auth, tiered quota enforcement, Alembic migrations, a monitoring dashboard, and CI. Published to PyPI with cross-platform installers.
- **What came of it:** installable by anyone via `pip install devmemory`; used across multiple AI clients. *Collect a real usage number before writing this beat* — PyPI download count is the obvious one.

Screenshot to capture: the monitoring dashboard, or a terminal showing memory being recalled in a second tool after being written in a first. The second is more convincing — it shows the actual claim.

Status chip: `<span class="status">Live · on PyPI</span>`

Reminder: monthly desktop notification, first Saturday, 10:00. Managed by cron — see `remind-next-case.sh` in this repo.

Evidence: reminder is set

`crontab -l` on my machine

```

# next-case-reminder
0 10 1-7 * 6 bash /home/commanderzero/projects/ys-home/remind-next-case.sh --fire

```

Verified working

Tested under a stripped environment (`env -i`, no `DISPLAY`, no `DBUS_SESSION_BUS_ADDRESS`) because that is what cron actually provides, and `notify-send` is a silent no-op without them:

```

$ env -i HOME="$HOME" PATH=/usr/bin:/bin bash remind-next-case.sh --now
Test notification sent.
exit: 0

```

Each fire also appends to `~/local/state/next-case-reminder.log`, so a missed month is visible rather than silent.

Build context preserved

I work in Claude Code rather than a Claude Project, so the equivalent of "keep the Project" is a committed `CLAUDE.md` in the site repo, which any future session loads automatically. It records the audience and the one action, the voice rules, the WCAG contrast requirement with a runnable check, and the gotchas already paid for: deploy `ys-home-dist` never `ys-home` (dragging the source folder once published `.git`), Netlify form detection being opt-in, and social preview caching. It is version-controlled and excluded from the deploy folder.